

MODUL 5

Ensemble Learning

Bagging, Boosting, dan Stacking

A. Tujuan Praktikum

1. Mahasiswa memahami konsep dasar Ensemble Learning dan berbagai pendekatannya (Bagging, Boosting, Stacking).
2. Mahasiswa mampu mengimplementasikan Random Forest, Gradient Boosting, AdaBoost, dan XGBoost menggunakan Python.
3. Mahasiswa dapat membandingkan performa model ensemble dengan model tunggal dan menganalisis feature importance.

B. Dasar Teori

Ensemble Learning adalah strategi machine learning yang menggabungkan prediksi dari beberapa model (learner) untuk menghasilkan prediksi akhir yang lebih akurat dan robust dibandingkan model tunggal manapun [1], [2]. Prinsip dasarnya berakar pada wisdom of crowds: gabungan pendapat banyak model yang beragam akan lebih akurat daripada satu model terbaik. Ada tiga pendekatan utama: Bagging (Bootstrap Aggregating) melatih beberapa model secara paralel pada subset data yang berbeda (bootstrap samples) dan menggabungkan prediksinya melalui voting (klasifikasi) atau averaging (regresi). Random Forest adalah implementasi bagging yang menggunakan Decision Tree sebagai base learner dengan tambahan pemilihan fitur acak pada setiap split, menghasilkan diversitas yang lebih tinggi antar tree [1].

Boosting adalah pendekatan sekuensial di mana setiap model baru difokuskan untuk memperbaiki kesalahan model sebelumnya. AdaBoost (Adaptive Boosting) memberikan bobot lebih tinggi pada sampel yang salah diklasifikasikan oleh model sebelumnya [2]. Gradient Boosting membangun model baru dalam arah negatif gradien fungsi loss, secara bertahap mengurangi residual. XGBoost (Extreme Gradient Boosting) adalah implementasi Gradient Boosting yang sangat dioptimalkan dengan regularisasi, penanganan missing values, dan pemrosesan paralel, menjadikannya salah satu algoritma terpopuler dalam kompetisi machine learning [1], [6]. Stacking (Stacked Generalization) menggunakan prediksi dari beberapa model (level-0) sebagai input untuk model meta (level-1) yang mempelajari cara terbaik menggabungkan prediksi tersebut [5].

C. Latihan Praktikum

Latihan 1 - Bagging dan Random Forest

Implementasi BaggingClassifier dan Random Forest, perbandingan dengan Decision Tree tunggal.

Python

```
# Latihan 1: Bagging dan Random Forest
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split,
cross_val_score from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (BaggingClassifier, RandomForestClassifier,
AdaBoostClassifier, GradientBoostingClassifier) from sklearn.metrics import
accuracy_score

# Dataset
bc = load_breast_cancer()
X, y = bc.data, bc.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Decision Tree Tunggal
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_acc = accuracy_score(y_test, dt.predict(X_test))

# Bagging
bag = BaggingClassifier(n_estimators=100, random_state=42)
bag.fit(X_train, y_train)
bag_acc = accuracy_score(y_test, bag.predict(X_test))

# Random Forest
rf = RandomForestClassifier(n_estimators=100,
random_state=42) rf.fit(X_train, y_train)
rf_acc = accuracy_score(y_test, rf.predict(X_test))

print(f'Decision Tree : {dt_acc:.4f}')
print(f'Bagging : {bag_acc:.4f}')
print(f'Random Forest : {rf_acc:.4f}')
```

Latihan 2 - Feature Importance Random Forest

Menganalisis feature importance dari model Random Forest.

Python

```
# Latihan 2: Feature Importance
importances = rf.feature_importances_
indices = np.argsort(importances)[::-1]

# Plot 15 fitur terpenting
top_n = 15
plt.figure(figsize=(10, 6))
plt.bar(range(top_n), importances[indices[:top_n]], color='#2E86AB',
alpha=0.8) plt.xticks(range(top_n),
    [bc.feature_names[i] for i in indices[:top_n]],
    rotation=45, ha='right', fontsize=9)
plt.xlabel('Fitur')
plt.ylabel('Importance Score')
plt.title('Top 15 Feature Importance - Random Forest')
plt.tight_layout()
plt.show()
```

```
print('Top 5 Fitur Terpenting:')
for i in range(5):
    print(f' {i+1}. {bc.feature_names[indices[i]]}:
    {importances[indices[i]]:.4f}')
```

Latihan 3 - AdaBoost dan Gradient Boosting

Implementasi dan perbandingan metode boosting: AdaBoost dan Gradient Boosting.

Python

```
# Latihan 3: AdaBoost dan Gradient Boosting

# AdaBoost
ada = AdaBoostClassifier(n_estimators=100, learning_rate=0.5,
random_state=42) ada.fit(X_train, y_train)
ada_acc = accuracy_score(y_test, ada.predict(X_test))

# Gradient Boosting
gb = GradientBoostingClassifier(n_estimators=100,
learning_rate=0.1, max_depth=3, random_state=42) gb.fit(X_train,
y_train)
gb_acc = accuracy_score(y_test, gb.predict(X_test))

# Learning Curve AdaBoost
ada_staged = list(ada.staged_predict(X_test))
staged_acc = [accuracy_score(y_test, p) for p in ada_staged]

plt.figure(figsize=(9, 5))
plt.plot(range(1, len(staged_acc)+1), staged_acc,
color='#E74C3C', lw=2, label='AdaBoost')
plt.axhline(y=rf_acc, color='#2E86AB', linestyle='--',
label=f'RF ({rf_acc:.3f})')
plt.xlabel('Jumlah Estimator')
plt.ylabel('Test Accuracy')
plt.title('Learning Curve AdaBoost')
plt.legend()
plt.tight_layout()
plt.show()

print(f'AdaBoost : {ada_acc:.4f}')
print(f'Gradient Boost : {gb_acc:.4f}')
```

Latihan 4 - Stacking Ensemble

Membangun model Stacking dengan beberapa base learner dan meta learner.

Python

```
# Latihan 4: Stacking Ensemble
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# Base learners
```

```

estimators = [
    ('rf', RandomForestClassifier(n_estimators=50, random_state=42)),
    ('gb', GradientBoostingClassifier(n_estimators=50, random_state=42)),
    ('svm', SVC(probability=True, random_state=42)),
    ('knn', KNeighborsClassifier(n_neighbors=7)),
]

# Meta learner

```

© 2026 Program Studi Teknik Informatika Halaman **24**
 Modul Praktikum Pembelajaran Mesin tiafadhilal@eng.uir.ac.id

```

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(max_iter=1000),
    cv=5
)
stack.fit(X_train, y_train)
stack_acc = accuracy_score(y_test, stack.predict(X_test))

# Ringkasan perbandingan
summary = {
    'Decision Tree': dt_acc, 'Random Forest': rf_acc,
    'AdaBoost': ada_acc, 'Gradient Boosting': gb_acc,
    'Stacking': stack_acc
}
df_summary = pd.DataFrame.from_dict(summary,
    orient='index', columns=['Accuracy'])
print(df_summary.sort_values('Accuracy', ascending=False).round(4).to_string())

```

D. Tugas

Tugas Modul 5

Gunakan dataset `load_digits` dari `sklearn`. Bangun dan bandingkan performa Random Forest, AdaBoost, Gradient Boosting, dan Stacking Ensemble. Lakukan hyperparameter tuning sederhana untuk Random Forest (variasikan `n_estimators` dan `max_depth`). Visualisasikan learning curve (train vs test score) untuk Random Forest. Analisis mengapa ensemble method umumnya menghasilkan performa lebih baik dari model tunggal.

© 2026 Program Studi Teknik Informatika Halaman **25**